

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 6: TRANSFORM-AND-CONQUER

(REPRESENTATION CHANGE-2_3 TREE)

Abdullah Bal, PhD

Objectives



Recap

Searching Problem
Binary Search Trees
AVL Trees



Representation Change



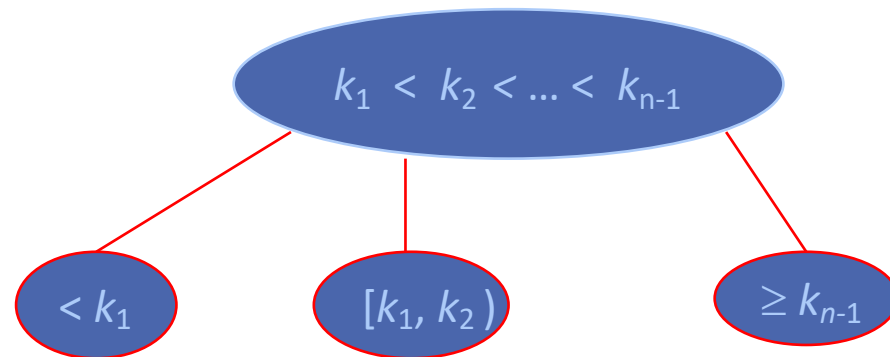
Multiway Search Trees



2-3 Trees

Multiway Search Trees

- The second idea of balancing a search tree is to allow **more than one key** in the same node of such a tree.
- A **multiway search tree** is a search tree that **allows more than one key** in the same node of the tree.
- A node of a search tree is called an ***n*-node** if it contains ***n*-1 ordered keys**
- It is divided the entire key range into *n* intervals pointed to by the node's *n* links to its children



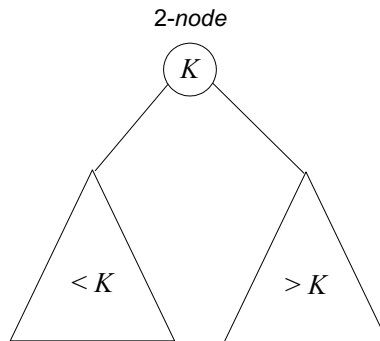
Note: Every node in a classical binary search tree is a 2-node

2-3 Tree

- The simplest implementation of multiway search trees is **2-3 trees**, introduced by the U.S. computer scientist John Hopcroft in 1970.
- A **2-3 tree** is a tree that can have nodes of two kinds:
 - 2-nodes
 - 3-nodes.

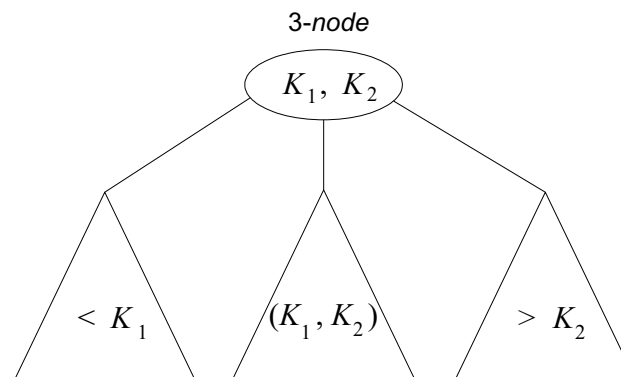
2-Node

- A **2-node** contains a single key K and has two children:
 - The left child serves as the root of a subtree whose keys are less than K ,
 - The right child serves as the root of a subtree whose keys are greater than K .
- A 2-node is the same kind of node we have in the classical binary search tree.



3-Node

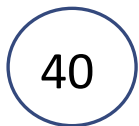
- A **3-node** contains
 - Two ordered keys K_1 and K_2 ($K_1 < K_2$)
 - Has three children.
- The leftmost child serves as the root of a subtree with keys less than K_1 ,
- The middle child serves as the root of a subtree with keys between K_1 and K_2 ,
- The rightmost child serves as the root of a subtree with keys greater than K_2 .



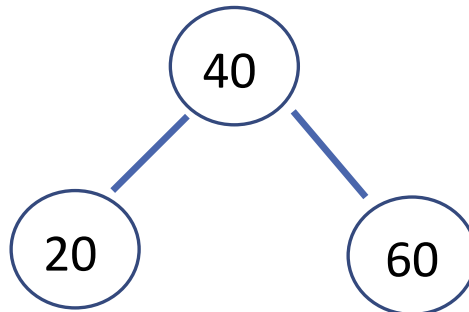
2-3 Tree Properties

1. Each node can either be a leaf node, 2- node, or 3-node
2. Keys is stored in **sorted order**.
3. It is a **balanced tree**.
4. All the leaf nodes are at **same level**.
5. Always **insertion** is **done at leaf**.

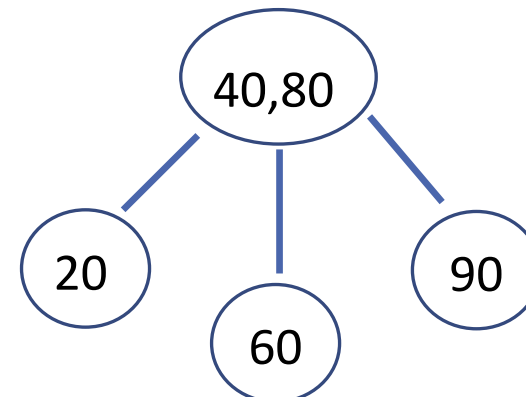
1 Leaf node



1 Key, 2 Children



2 Keys, 3 Children



2-3 Tree Searching

Searching:

- We start at the root.
 - If the root is a 2-node, we act as if it were a **binary search** tree:
 - We either stop if K is equal to the root's key or continue the search in the left or right subtree if K is, respectively, smaller or larger than the root's key.
 - If the root is a 3- node, we know after no more than two key comparisons whether the search can be stopped (if K is equal to one of the root's keys) or in which of the root's three subtrees it needs to be continued.

○

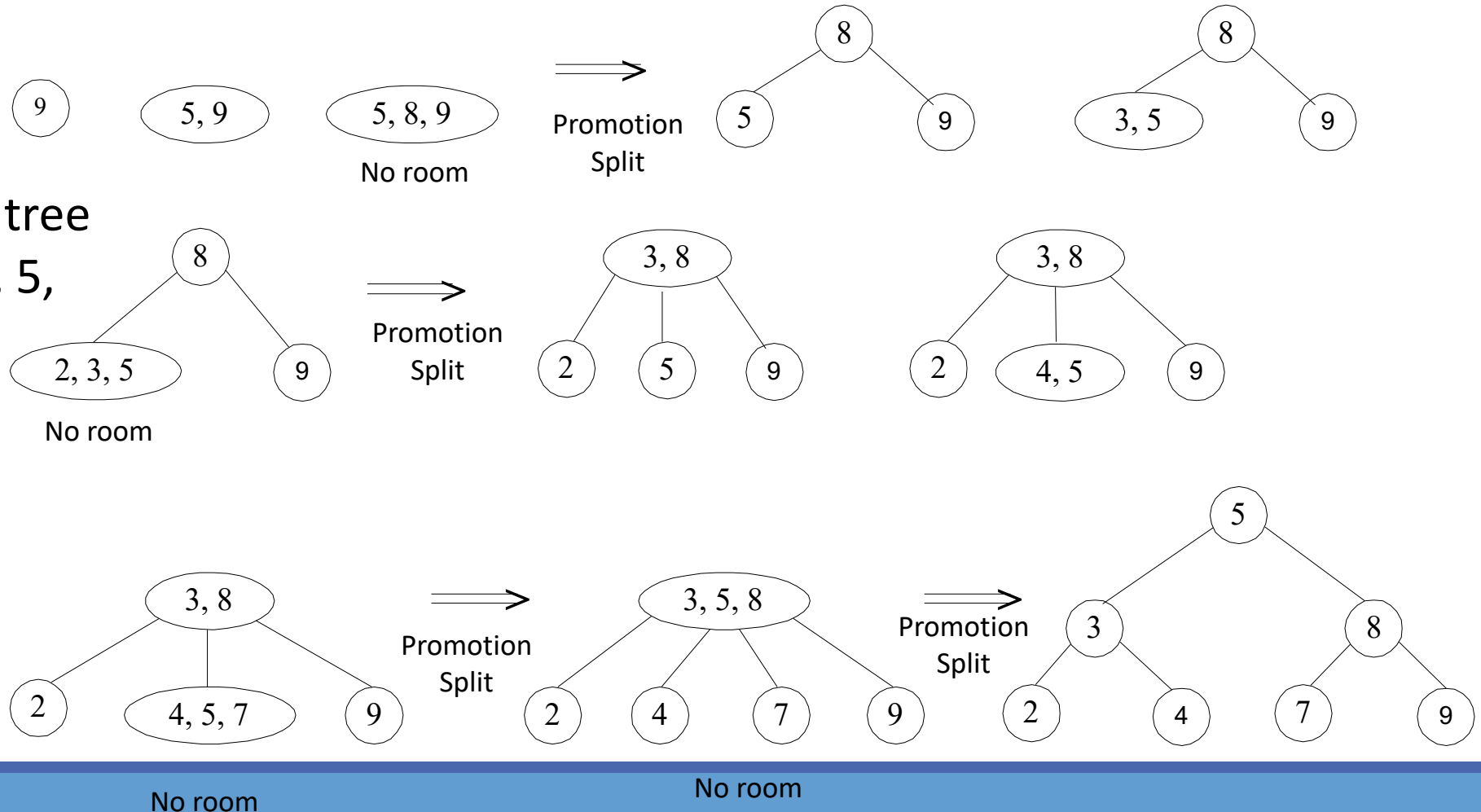
2-3 Tree Inserting

Inserting:

- We always insert a new key K in a leaf, except for the empty tree.
- The appropriate leaf is found by performing a search for K.
 - If the leaf in question is a 2-node, we insert K there as either the first or the second key, depending on whether K is smaller or larger than the node's old key.
 - If the leaf is a 3-node, we split the leaf in two:
 - The smallest of the three keys (two old ones and the new key) is put in the first leaf, the largest key is put in the second leaf, and the middle key is promoted to the old leaf's parent.
 - If the leaf happens to be the tree's root, a new root is created to accept the middle key.
 - Note that promotion of a middle key to its parent can cause the parent's overflow (if it was a 3-node) and hence can lead to several node splits along the chain of the leaf's ancestors.

Example 1: 2-3 tree construction

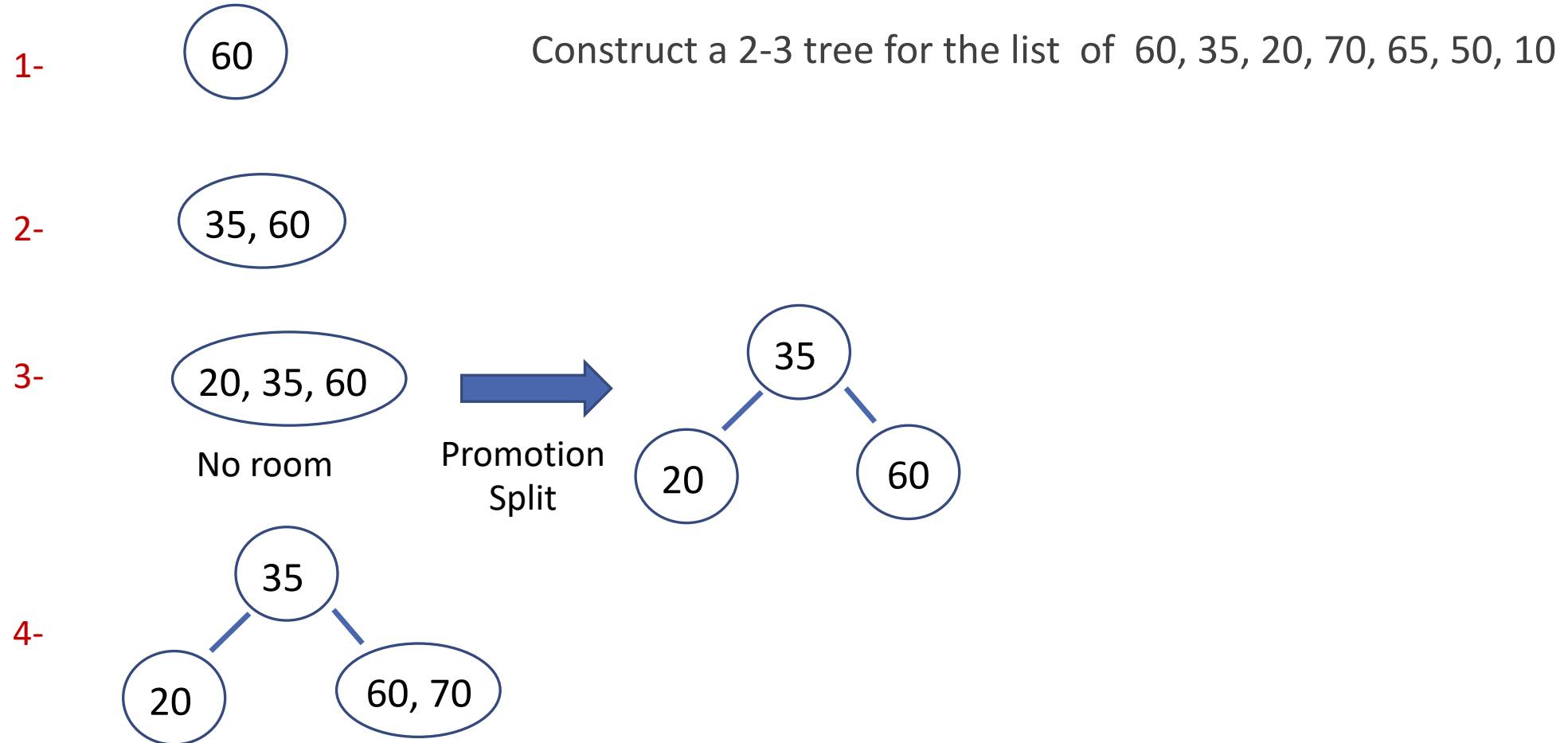
- Construct a 2-3 tree for the list of 9, 5, 8, 3, 2, 4, 7



Example 2: 2-3 tree construction

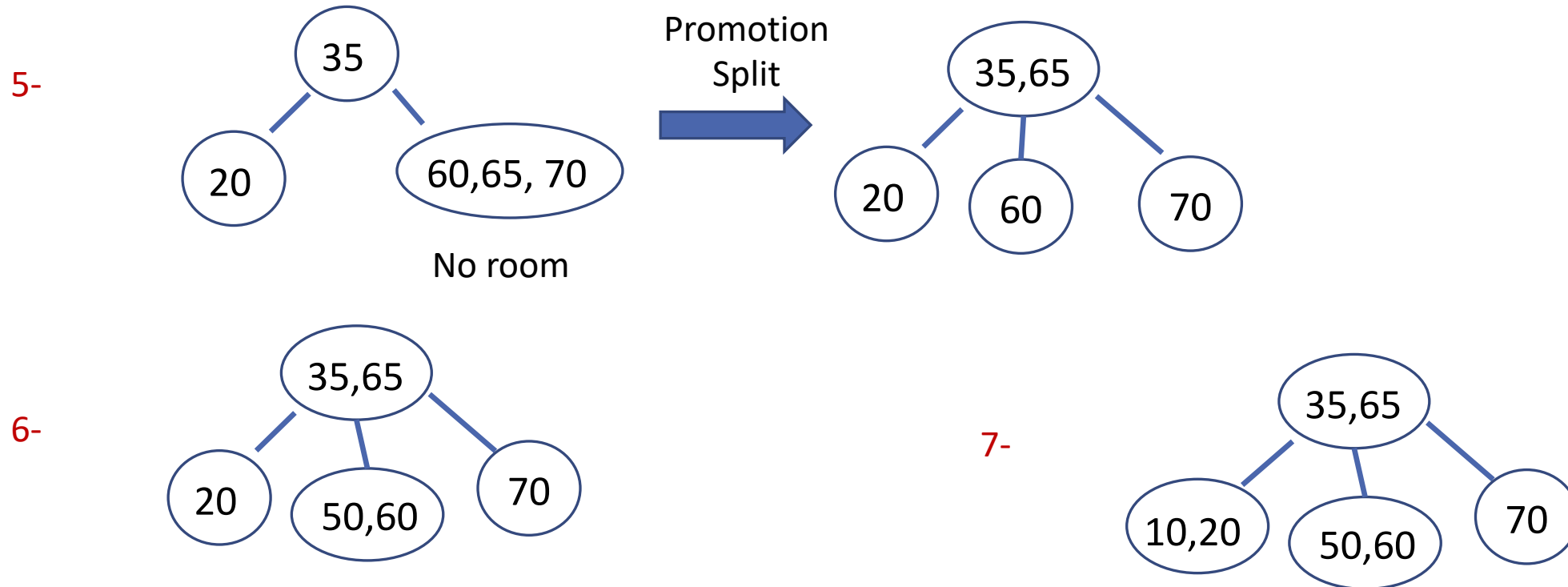
Construct a 2-3 tree for the list of 60, 35, 20, 70, 65, 50, 10

Example 2: 2-3 tree construction



Example 2: 2-3 tree construction

Construct a 2-3 tree for the list of 60, 35, 20, 70, 65, 50, 10



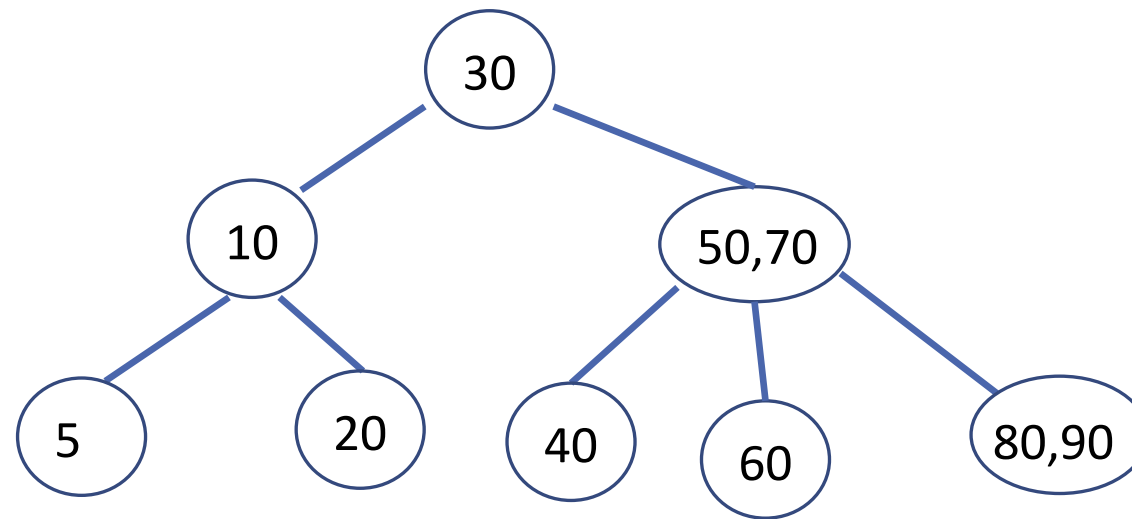
Example 3: 2-3 tree construction

Construct a 2-3 tree for the list of 40, 50, 60, 30, 20, 10, 5, 70, 80, 90

Example 3: 2-3 tree construction

Construct a 2-3 tree for the list of 40, 50, 60, 30, 20, 10, 5, 70, 80, 90

Solution:



Analysis of 2-3 trees

- As for any search tree, the efficiency of the dictionary operations depends on the **tree's height**.
- A 2-3 tree of height h with the smallest number of keys is a full tree of 2-nodes (such as the final tree in the previous figure for $h = 2$).
- Therefore, for any 2-3 tree of height h with n nodes, we get the inequality

$$n \geq 1 + 2 + \cdots + 2^h = 2^{h+1} - 1,$$

$$h \leq \log_2(n + 1) - 1.$$

- On the other hand, a 2-3 tree of height h with the largest number of keys is a full tree of 3-nodes, each with two keys and three children.

$$n \leq 2 \cdot 1 + 2 \cdot 3 + \cdots + 2 \cdot 3^h = 2(1 + 3 + \cdots + 3^h) = 3^{h+1} - 1$$

- Therefore, for any 2-3 tree with n nodes, $h \geq \log_3(n + 1) - 1.$

Analysis of 2-3 trees

- These lower and upper bounds on height h ,

$$\log_3(n + 1) - 1 \leq h \leq \log_2(n + 1) - 1,$$

- The time efficiencies of searching, insertion, and deletion are all in $\Theta(\log n)$ in both the worst and average case.

Generalization of 2-3 trees

- The idea of 2-3 tree can be generalized by allowing more keys per node
 - 2-3-4 trees
 - B-trees

Summary

- Representation Change
- Multiway search trees
- 2-3 Trees

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.